Observability

你不能修復你看不見的東西

系統上線之後，真正的挑戰才開始。

你精心設計了架構、寫了測試、做了 code review，然後部署到生產環境。某個週五晚上，用戶開始回報「結帳失敗」。你打開 dashboard，只看到一個服務的回應時間從 200ms 跳到了 8 秒。是資料庫慢了？是某個下游 API 掛了？是新部署的程式碼有 bug？是流量突然暴增？你不知道。你只知道某個地方出了問題，但你像在黑暗中摸索。

這就是可觀測性（Observability）存在的原因：讓你能夠從系統的外部輸出，比如數字、文字、追蹤記錄，來推斷系統的內部狀態。

可觀測性建立在三個支柱上：**Metrics（指標）**、**Logs（日誌）**、**Traces（追蹤）**。它們不是同一件事的三個說法，而是各自回答不同的問題，相互補充。

Metrics（指標）

Metrics 回答的問題是：**系統現在的狀態如何？趨勢是什麼？**

Metrics 是數值型的時序資料，每隔一段時間（例如每 15 秒）記錄一個數值，長時間下來形成曲線。你可以在上面做聚合、計算百分位數、設定閾值觸發警告。

Metrics 的類型

Counter（計數器）：只會單調遞增的數值。HTTP 請求總數、錯誤總數、發送的訊息數。它本身不太有意義，有意義的是它的**增長速率，像是**每秒新增多少請求、每分鐘新增多少錯誤。

Gauge（儀表）：可以上下浮動的數值，代表某個時間點的即時狀態。目前的記憶體使用量、連線池的使用數、佇列的深度。

Histogram（直方圖）：把數值分桶統計分佈，最常用來計算延遲的百分位數。「P99 延遲是 450ms」就是從 Histogram 計算出來的，也就是把所有請求的延遲放進桶裡，排在第 99 百分位的那個值。

```
from prometheus_client import Counter, Gauge, Histogram

# Counter：請求總數
REQUEST_COUNT = Counter(
    'http_requests_total',
```

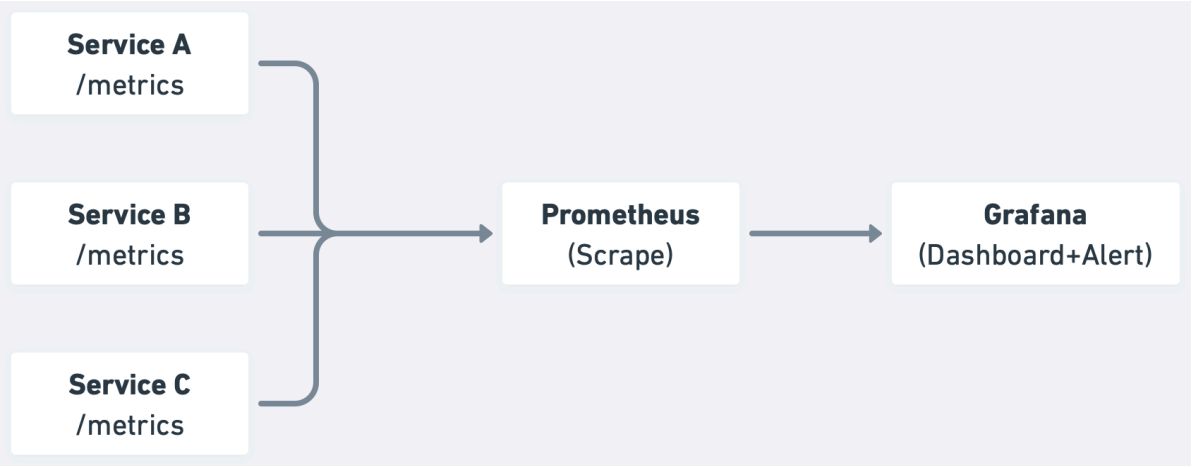
buildnoat.org

```
    'Total HTTP requests',
    ['method', 'endpoint', 'status_code']
)

# Gauge：目前的連線數
ACTIVE_CONNECTIONS = Gauge(
    'active_connections',
    'Number of active connections'
)

# Histogram：請求延遲分佈
REQUEST_LATENCY = Histogram(
    'http_request_duration_seconds',
    'HTTP request latency',
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)

# 使用
@app.route('/orders')
def get_orders():
    with REQUEST_LATENCY.time():
        REQUEST_COUNT.labels(method='GET', endpoint='/order
s', status_code=200).inc()
        ACTIVE_CONNECTIONS.inc()
        result = fetch_orders()
        ACTIVE_CONNECTIONS.dec()
        return result
```

pulumoat.org

Prometheus + Grafana

Prometheus 是最主流的開源 Metrics 系統。它用 Pull 模式定期從各個服務的 `/metrics` 端點拉取數據，存入時序資料庫。**Grafana** 則是在 Prometheus 的數據上建立視覺化儀表板和警告規則。



你應該監控哪些 Metrics？

Google 的 SRE 書提出了 **「四個黃金信號」**，是任何服務都應該監控的最基本指標：

- **Latency（延遲）**：請求需要多少時間？要區分成功請求和失敗請求的延遲，比如一個快速失敗的錯誤和一個緩慢成功的請求，意義完全不同
- **Traffic（流量）**：每秒有多少請求？這是衡量系統負載的基準
- **Errors（錯誤率）**：請求失敗的比例是多少？區分顯性錯誤（HTTP 500）和隱性錯誤（回傳 200 但內容是錯的）
- **Saturation（飽和度）**：系統還有多少餘裕？CPU 使用率、記憶體使用率、磁碟空間，越接近上限，系統越脆弱

警告：對症狀警告，而不是對原因

一個常見的錯誤是「對原因警告」：CPU 超過 80% 就警告、記憶體超過 70% 就警告。這樣做的結果是警告太多，但不一定代表用戶真的有感受到問題，警告疲勞（alert fatigue）會讓工程師開始忽略警告。

更好的做法是**對症狀警告**：用戶有沒有感受到問題？

```
# 不好：對原因警告
- alert: HighCPU
  expr: cpu_usage > 0.8
  # CPU 高不一定代表用戶有問題

# 好：對症狀警告
- alert: HighErrorRate
  expr: rate(http_requests_total{status_code=~"5.."}[5m]) >
0.01
```

mildnoal.org

```
  annotations:
    summary: "錯誤率超過 1%，用戶正在受到影響"

- alert: HighLatency
  expr: histogram_quantile(0.99, http_request_duration_seco
nds) > 1.0
  annotations:
    summary: "P99 延遲超過 1 秒"
```

Logs（日誌）

Logs 回答的問題是：**某個特定事件到底發生了什麼？**

當警告觸發，你知道「錯誤率上升了」，但你不知道**為什麼**。這時候你需要 Logs，每個事件的詳細文字記錄，讓你能夠追查具體的失敗原因。

結構化日誌（Structured Logging）

不要用純文字 Log，用 JSON 格式的**結構化日誌**。純文字 Log 很難用程式解析和查詢；結構化日誌讓你能用欄位過濾、聚合、搜尋。

```
import structlog

logger = structlog.get_logger()

# 不好：純文字
# logger.info("User 123 created order 456 for $100")

# 好：結構化
logger.info(
    "order_created",
    user_id=123,
    order_id=456,
    amount=100,
    payment_method="credit_card",
    duration_ms=45
)
```

輸出：

buildmoat.org

```
{
  "event": "order_created",
  "user_id": 123,
  "order_id": 456,
  "amount": 100,
  "payment_method": "credit_card",
  "duration_ms": 45,
  "timestamp": "2024-01-15T10:23:45Z",
  "service": "order-service",
  "level": "info"
}
```

Log Level

不同嚴重程度的事件用不同的 level，讓你能快速過濾：

- **DEBUG**：開發時用的詳細資訊，生產環境通常關閉
- **INFO**：正常的業務事件（訂單建立、用戶登入）
- **WARN**：值得注意但不緊急的狀況（重試成功、快取 miss 率偏高）
- **ERROR**：某個操作失敗了，但系統還在運作
- **FATAL**：系統無法繼續運作，需要立即處理

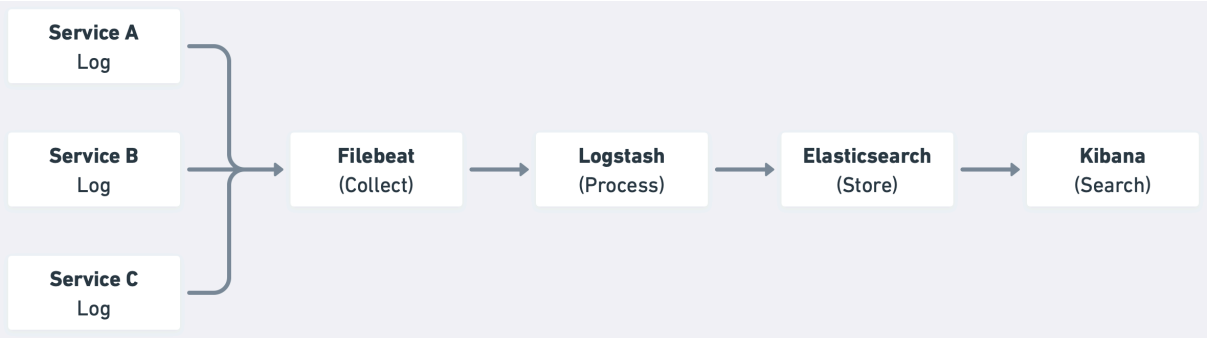
集中式日誌（Centralized Logging）

每個服務都有自己的 Log，如果分散在各台機器上，出問題時要一台一台 SSH 進去查，效率極差。集中式日誌把所有服務的 Log 匯集到同一個地方。

ELK Stack 是最常見的選擇：

- **Elasticsearch**：儲存和搜尋 Log
- **Logstash**（或 Filebeat）：收集和傳送 Log
- **Kibana**：視覺化查詢介面

buildmoat.org



現代的替代方案是 **Grafana Loki**，設計更輕量，不對 Log 內容建全文索引（只對標籤建索引），儲存成本更低。

取樣（Sampling）

一個高流量的服務每秒可能產生幾十萬條 Log，全部儲存的成本極高。可以對 INFO level 的 Log 做取樣（例如只保留 10%），但 ERROR level 的 Log 永遠 100% 保留。

Traces（分散式追蹤）

Traces 回答的問題是：**一個請求在整個系統裡走了哪些路徑？每一步花了多久？**

在微服務架構裡，一個用戶請求可能經過十幾個服務。當這個請求變慢了，Metrics 告訴你「P99 延遲上升了」，Logs 告訴你各個服務各自發生了什麼，但你沒辦法把它們串起來，看到「這個特定請求從 API Gateway 進來、打了 User Service、再打了 Order Service、在 Order Service 等資料庫等了 2 秒、然後回到 API Gateway 回傳結果」這樣完整的路徑。

Traces 解決的就是這個問題。

Trace 和 Span

一個 **Trace** 代表一個請求的完整生命週期，由多個 **Span** 組成。每個 Span 代表一個操作單元：調用一個服務、執行一個資料庫查詢、發一個 HTTP 請求。

```
Trace ID: abc-123
|
├── Span: API Gateway (0ms ~ 250ms)
|   ├── Span: Auth Service (5ms ~ 15ms)
|   └── Span: Order Service (20ms ~ 240ms)
|       ├── Span: DB Query - SELECT orders (25ms ~ 180ms)
← 瓶頸在這裡
|       └── Span: Notification Service (185ms ~ 200ms)
```

buildmvt.org

這張圖讓你立刻看出：整個請求花了 250ms，其中資料庫查詢占了 155ms，這是需要優化的地方。

Trace Context 的傳遞

要讓 Trace 跨服務工作，每個請求都需要帶著一個 **Trace ID** 往下傳。當服務 A 呼叫服務 B，它在 HTTP header 裡附上 Trace ID；服務 B 看到這個 ID，知道自己是哪個 Trace 的一部分，就在同一個 Trace 下建立新的 Span。

```
# OpenTelemetry 是現在最主流的標準
from opentelemetry import trace
from opentelemetry.propagate import inject, extract

tracer = trace.get_tracer(__name__)

def create_order(request):
    # 從上游取得 trace context
    context = extract(request.headers)

    with tracer.start_as_current_span("create_order", conte
xt=context) as span:
        span.set_attribute("user_id", request.user_id)
        span.set_attribute("amount", request.amount)

        # 呼叫下游服務時，自動注入 trace context 到 header
        headers = {}
        inject(headers)
        response = downstream_service.call(headers=headers)

        span.set_attribute("order_id", response.order_id)
        return response
```

OpenTelemetry 已經成為業界標準，它定義了統一的 API 和資料格式，讓你可以自由選擇後端（Jaeger、Zipkin、Grafana Tempo、Datadog 都支援）。

三者如何互補

Metrics、Logs、Traces 各自有擅長和不擅長的事：

buildmoat.org

	Metrics	Logs	Traces
回答的問題	系統狀態怎樣？	這個事件發生了什麼？	這個請求走了什麼路徑？
資料形式	數值 + 時序	文字事件	有向無環圖（DAG）
適合做	警告、趨勢分析	Debug 特定錯誤	找出跨服務的延遲瓶頸
儲存成本	低	高	中
查詢速度	快	較慢	中

實際的排查流程通常是這樣串起來的：

```
1. Metrics 警告觸發
   "P99 延遲從 200ms 升到 2 秒"
       ↓
2. 查 Traces 找到慢的請求
   "這些慢請求都卡在 Order Service 的 DB Query"
       ↓
3. 查 Logs 找到具體原因
   "Order Service 在這段時間有大量 'slow query: 1.8s' 的 warni
ng log"
       ↓
結論：Order Service 有個 SQL 查詢沒有用到索引
```

SLI、SLO、SLA

可觀測性的最終目的是讓你能夠衡量和保證**服務的可靠性**。這裡有三個常被混淆的概念：

SLI（Service Level Indicator）：你用來衡量服務品質的具體指標。例如「成功請求的比例」、「P99 延遲」。SLI 就是一個 Metric。

SLO（Service Level Objective）：你對 SLI 設定的目標值。例如「成功率 ≥ 99.9%」、「P99 延遲 ≤ 500ms」。SLO 是你對自己的承諾，是決定是否要觸發警告、是否要放慢發布速度的依據。

SLA（Service Level Agreement）：你對外部客戶承諾的合約，通常比內部的 SLO 寬鬆。如果你的內部 SLO 是 99.9%，你對外的 SLA 可能是 99.5%留下緩衝空間。

```
SLI：過去 30 天，成功請求 / 總請求 = 0.9995 (99.95%)
SLO：成功率需要 ≥ 99.9%
SLA：對客戶承諾 99.5%（對外合約）
```

buildmoat.org

```
目前狀態：SLI > SLO > SLA，一切正常
```

Error Budget（錯誤預算）

SLO 給了你一個可以「消耗」的**錯誤預算**。如果你的 SLO 是 99.9%，那你一個月允許的停機時間是 0.1% × 30天 × 24小時 x 60分鐘 = 約 43 分鐘。

這個框架讓「可靠性」和「功能開發速度」之間的取捨變得清晰：

- 錯誤預算還充足 → 可以繼續快速發布新功能
- 錯誤預算快耗盡 → 需要放慢發布節奏，優先修復可靠性問題

什麼時候在面試裡用這些

主動說明可觀測性設計

設計任何生產系統時，主動說明你怎麼知道它運作正常：

「這個服務我會暴露 Prometheus metrics，監控四個黃金信號：延遲（P99）、流量（RPS）、錯誤率（5xx 比例）、飽和度（CPU 和記憶體使用率）。所有服務的 Log 集中到 ELK，用結構化 JSON 格式，方便查詢。跨服務的請求用 OpenTelemetry 做 distributed tracing，送到 Jaeger。SLO 設定成功率 99.9%、P99 延遲 500ms 以內。」

常見面試情境

設計 URL Shortener：「縮短後的 URL 每次被訪問，我會記錄一個 Counter metric。Grafana 上可以看到每個短網址的訪問趨勢，也能在訪問量異常（遠超平均）時觸發警告，可能代表某個連結被病毒式傳播，需要擴容。」

設計通知系統：「訊息從進入佇列到用戶收到推播，我會在每個階段記錄時間戳記，用 Trace 把整條鏈路串起來。如果 P99 的端到端延遲超過 30 秒，觸發警告。Metrics 上同時監控每個渠道（push、email、SMS）的發送成功率，任一渠道的成功率低於 95% 就警告。」

設計電商訂單系統：「訂單金額是業務關鍵指標，我會用 Gauge 記錄每分鐘的 GMV（總成交金額）。如果 GMV 在短時間內異常下跌超過 20%，立即觸發警告，這比等用戶回報「無法結帳」要快得多。」

常見的 Deep Dive 問題

「Metrics、Logs、Traces 分別在什麼時候用？」

buildroot.org

「三者回答不同的問題，實際使用是有順序的。**Metrics** 是警告的基礎，讓你第一時間知道「有問題」，但不告訴你為什麼，它是觀察系統狀態的儀表板。問題被發現後，**Traces** 幫你縮小範圍，在哪個服務、哪個操作出了問題。找到瓶頸之後，**Logs** 給你細節，那個操作具體發生了什麼錯誤或異常狀況。三者搭配才是完整的排查流程：Metrics 警告 → Traces 定位 → Logs 確診。」

「你怎麼設定 SLO？」

「SLO 的設定要從用戶的角度出發，而不是從系統能力出發。先問：用戶能接受的最差體驗是什麼？然後把這個翻譯成可測量的 SLI，再設定稍微嚴格一點的 SLO，讓工程團隊在 SLA 違反之前就有預警。

另一個重要原則是：SLO 不應該是 100%。100% 的 SLO 意味著你永遠不能做任何可能影響穩定性的改動，包括發布新功能。99.9% 給了你每個月 43 分鐘的錯誤預算，讓你能夠在這個預算內快速迭代。SLO 設得太高，工程師會花大量時間在可靠性上而犧牲功能開發；太低，用戶體驗就差了。通常從 99.9% 開始，根據業務敏感度調整。」

「Log 量太大怎麼辦？」

建議策略：

取樣（Sampling）：對 INFO level 的 Log 只保留 10-20%，ERROR 和 WARN 保留 100%。

動態 Log level：生產環境平時只開 WARN 以上，出問題時動態把某個服務的 level 調成 DEBUG，查完再調回去。

設定 TTL：Log 不需要永久保留，通常 30 到 90 天就夠，舊的自動刪除。

結構化 + 建索引：不對全文建索引，只對關鍵欄位（service、user_id、trace_id、level）建索引，查詢速度快、儲存成本低。

總結

可觀測性是讓系統「透明」的工程能力，不是防止故障，而是確保故障發生時你能快速看見、快速診斷、快速修復。

Metrics：數值型時序資料，回答「系統狀態如何」，是警告的基礎；監控四個黃金信號（延遲、流量、錯誤率、飽和度）

- **Logs**：事件的詳細文字記錄，回答「這個事件發生了什麼」；用結構化日誌、集中存放
- **Traces**：請求跨服務的完整路徑，回答「這個請求走了哪裡、哪裡慢了」；用 OpenTelemetry 做標準化

buildnoat.org

- **SLI / SLO / SLA**：把可靠性量化，讓「夠不夠可靠」變成可以討論和決策的數字；用 Error Budget 平衡可靠性和開發速度

在面試裡，不需要把三個支柱都詳細展開，而是說明你**設計了什麼指標**來衡量系統是否健康，以及**出問題時你怎麼排查，當**你在畫架構圖的同時，也正在思考它跑起來之後要怎麼維運。

buildmoat.org